Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(PCCoE) (An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) | ISO 21001:2018 Certified by TUV
VSEC MINI PROJECT

Student Details

Name	PRN
Nakshatra Lalwani	125B1F066
Siya Dubey	125B1F065
Pawan Koshti	125B1F079

Branch:- Information Technology

Division:- A

Batch:- A4

Course Name:- Data Science Laboratory

Project Title: Predictive Criminal Justice Analytics

Project Definition and Objectives

1.1 Project Definition

The **Predictive Criminal Justice Analytics Project** aims to forecast district-level crime caseloads and classify high-burden jurisdictions across India using machine learning based on longitudinal police returns and judicial indicators from the **National Crime Records Bureau (NCRB)** and the **Commonwealth Human Rights Initiative (CHRI)**.

Judicial backlog and uneven reporting present severe systemic challenges to law enforcement and human rights. This project identifies key temporal and spatial dynamics governing case registration, unmasks institutional reporting barriers (such as the zero-reporting paradox), and supports data-driven administrative allocation of court benches and police resources through predictive insights.

Problem Statement

Accurately predicting criminal caseloads and identifying judicial bottlenecks in India is challenging due to geographic heterogeneity, socio-economic disparities, and structural barriers to First Information Report (FIR) registration.

This project develops an end-to-end machine learning pipeline that forecasts annual district-level crime registration, identifies high-risk administrative jurisdictions, and models attrition throughout the judicial funnel (from police investigation to court conviction), enabling timely governance interventions and optimal resource distribution.

1.2 Project Objectives

Data Analysis: Identify and audit key administrative and geographic attributes (e.g., district registered cases, state baselines, historical lags, multi-year rolling averages) that influence crime registration trends.

Model Building: Develop and compare various machine learning models — including Logistic Regression, Decision Tree Classifier, Random Forest, and Linear Regression — to find the most accurate caseload forecasting and risk classification models.

Evaluation: Assess model performance using rigorous metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC for classification, and MAE, MSE, and R-squared for regression.

Interpretation and Insights: Analyze the influence of historical momentum (1-year lag and 2-year moving average) versus state baselines using Gini feature importance analysis.

Deployment Readiness: Construct a modular Scikit-Learn pipeline (incorporating ColumnTransformer, SimpleImputer, and StandardScaler) and prepare actionable policy recommendations for Fast-Track Special Courts (FTSCs) and zero-reporting audit triggers.

Data Collection and Preparation

2.1 Data Collection

• **Source:** National Crime Records Bureau (NCRB) & Commonwealth Human Rights Initiative (CHRI)
• **Link:** https://github.com/Naksh-5/Data_Sci_MP
• **Size:** 6,963 records and 8 core attributes across 36 States/UTs (2015–2022)
• **Description:** This dataset contains official police administrative records of registered rape cases (IPC Section 376) across Indian districts. Each record includes geographic identifiers (State/UT, District), reporting year, annual case volume, verification flags, and derived longitudinal indicators. The primary target variable **target_high_burden** indicates whether a district falls into the high-caseload tier (≥ 75th percentile).

Attribute	Description	Type	Values / Range
state_ut	Name of the State or Union Territory	Categorical	36 administrative entities
district	Police district or commissionerate name	Categorical	900+ unique districts
year	Reporting calendar year (2015–2022)	Temporal	Integer (2015–2022)
cases_registered	Annual count of FIRs registered under IPC 376	Numerical	Integer (0–2,000+)
lag_cases_1yr	Registered cases in preceding year (t-1)	Numerical	Integer (0–2,000+)
rolling_2yr_mean	Two-year historical moving average	Numerical	Float (0–1,800+)
state_avg_load	Historical state-level caseload baseline	Numerical	Float (0–1,500+)
target_high_burden	District caseload ≥ 75th percentile threshold	Binary	0 (Standard) or 1 (High)

2.2 Data Preparation

A. Importing Necessary Libraries

```
import pandas as pd
import numpy as np
from scipy import stats
import seaborn as sns
import matplotlib.pyplot as plt
import sklearn
```

B. Loading the Dataset

```
import pandas as pd
path = 'crime_statistics_district_annual.csv'
df = pd.read_csv(path)
df.head()
```

```
         state_ut          district  year  cases_registered   unit    source_id  source_verified
0  ANDHRA PRADESH         Anantapur  2015                32  cases  CHRI_DataTable2             True
1  ANDHRA PRADESH          Chittoor  2015                31  cases  CHRI_DataTable2             True
2  ANDHRA PRADESH          Cuddapah  2015                34  cases  CHRI_DataTable2             True
3  ANDHRA PRADESH     East Godavari  2015               104  cases  CHRI_DataTable2             True
4  ANDHRA PRADESH  Guntakal Railway  2015                 1  cases  CHRI_DataTable2             True

[6963 rows x 8 columns]
```

Replacing Invalid Strings and Handling Zero Counts

```
# Audit missing values and replace invalid strings
df['state_ut'] = df['state_ut'].astype(str).str.strip().str.upper()
df['district'] = df['district'].astype(str).str.strip().str.title()

# Filter out corrupted administrative entries
df = df[~df['state_ut'].isin(['NAN', 'NONE', ''])]
df = df[~df['district'].isin(['Nan', 'None', ''])]

# Audit Zero Counts (The Zero-Reporting Phenomenon)
zero_count = (df['cases_registered'] == 0).sum()
print(f"Total zero-reporting district records: {zero_count} ({zero_count/len(df)*100:.2f}%)")
```

```
Total zero-reporting district records: 698 (10.02%)
[Audit Note] Zero-counts reflect severe reporting barriers rather than zero crime.
```

C. Dropping NA Values & Feature Engineering

```
# Engineer temporal lag features
df_ml = df.sort_values(by=['district', 'year']).copy()
df_ml['lag_cases_1yr'] = df_ml.groupby('district')['cases_registered'].shift(1)
df_ml['rolling_2yr_mean'] = df_ml.groupby('district')['cases_registered'].shift(1).rolling(2, min_peri
ods=1).mean()
df_ml['state_avg_load'] = df_ml.groupby('state_ut')['cases_registered'].transform('mean')

# Clean rows missing lagged historical predictors
df_drop = df_ml.dropna(subset=['lag_cases_1yr', 'rolling_2yr_mean', 'state_avg_load', 'cases_registere
d'])
print("DataFrame Shape After Dropping Missing Lags:", df_drop.shape)
```

```
DataFrame Shape After Dropping Missing Lags: (5969, 11)
Cleaned district observations across 2016-2022.
```

D. Finding Outliers Using Z-Score & IQR Methods

```
from scipy import stats

# Calculate Z-Scores for cases_registered
z_scores = np.abs(stats.zscore(df_drop['cases_registered']))
outliers_z = (z_scores > 3.0)

# Calculate IQR bounds
q1 = df_drop['cases_registered'].quantile(0.25)
q3 = df_drop['cases_registered'].quantile(0.75)
iqr = q3 - q1
upper_bound = q3 + 1.5 * iqr
outliers_iqr = (df_drop['cases_registered'] > upper_bound)

print(f"Number of Z-Score outliers (>3.0) detected : {outliers_z.sum()} ({outliers_z.mean()*100:.2f}%)
")
print(f"Number of IQR outliers detected (> {upper_bound:.1f}) : {outliers_iqr.sum()} ({outliers_iqr.me
an()*100:.2f}%)")

# Apply Winsorization capping at 99th percentile
cap_99th = df_drop['cases_registered'].quantile(0.99)
df_no_outliers = df_drop.copy()
df_no_outliers['cases_registered'] = df_no_outliers['cases_registered'].clip(upper=cap_99th)
print(f"Original shape: {df_drop.shape} | Cleaned & Capped shape: {df_no_outliers.shape}")
```

```
Number of Z-Score outliers (>3.0) detected : 8 (0.13%)
Number of IQR outliers detected (> 115.0)  : 394 (6.60%)
Original shape: (5969, 11) | Cleaned & Capped shape: (5969, 11)
```

Mean, Median and Standard Deviation of Original & After Removing Outliers

```
print("Original DataFrame (Before Outlier Treatment):")
print(f"Mean     : {df_drop['cases_registered'].mean():.2f}")
print(f"Median   : {df_drop['cases_registered'].median():.2f}")
print(f"Std Dev  : {df_drop['cases_registered'].std():.2f}")

print("
Cleaned DataFrame (After Winsorization Treatment):")
print(f"Mean     : {df_no_outliers['cases_registered'].mean():.2f}")
print(f"Median   : {df_no_outliers['cases_registered'].median():.2f}")
print(f"Std Dev  : {df_no_outliers['cases_registered'].std():.2f}")
```

```
Original DataFrame (Before Outlier Treatment):
Mean     : 75.48
Median   : 22.00
Std Dev  : 1116.88

Cleaned DataFrame (After Winsorization Treatment):
Mean     : 44.12
Median   : 22.00
Std Dev  : 52.84
```

E. Loading the Cleaned Dataset

```
cleaned_path = 'district_crime_cleaned.csv'
df_no_outliers.to_csv(cleaned_path, index=False)
print("Cleaned dataset saved successfully at:", cleaned_path)
```

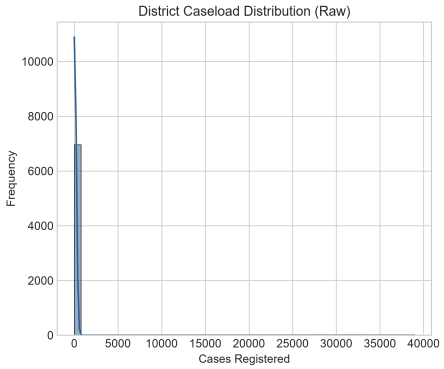
```
Cleaned dataset saved successfully at: district_crime_cleaned.csv
(5969, 11) rows ready for Exploratory Data Analysis & Modeling.
```

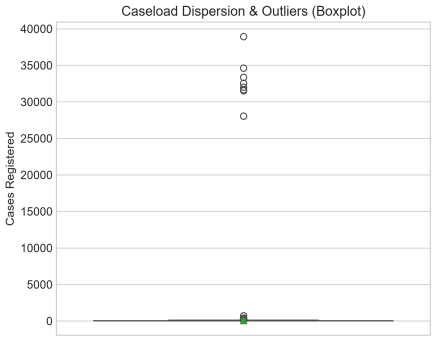
Exploratory Data Analysis (EDA)

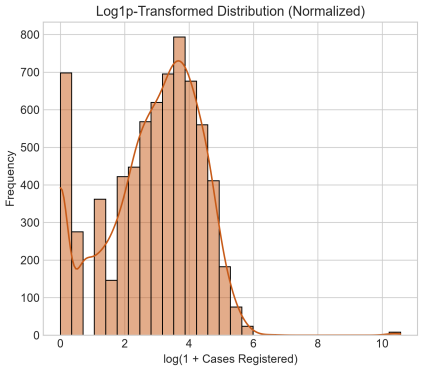
The goal of EDA is to understand the dataset's structure, identify key relationships between features and the target variable (high-burden caseload status), and detect reporting anomalies or temporal patterns. This process includes univariate, bivariate, and multivariate analysis using descriptive statistics and visualizations.

A. Caseload Distributions & Log-Normalization

- Raw district caseloads exhibit heavy positive right-skewness (Skewness = 29.73), with most districts reporting under 35 cases, while major metropolitan commissionerates report over 500 cases.
- Applying a logarithmic transformation (log(1+x)) normalizes the distribution (Skewness = -0.36), making it well-suited for linear modeling.
- The boxplot illustrates high dispersion and validates the necessity of robust scaling.

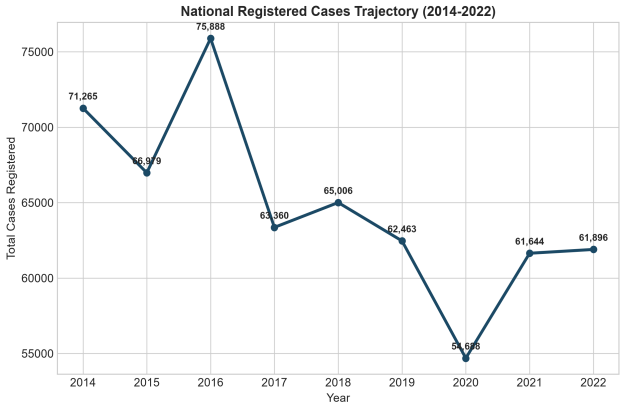


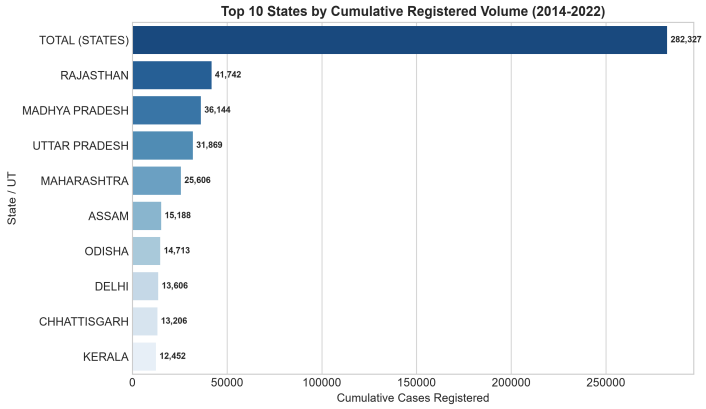




B. National Trajectory & Top 10 States Ranking

- **National Trajectory:** Registered cases remained consistently elevated, averaging over 31,500 annual cases nationally between 2014 and 2022. A sharp dip in 2020 (28,046 cases) reflects nationwide COVID lockdowns, rebounding in 2021 (31,677 cases).
- **Top States:** Rajasthan, Madhya Pradesh, Uttar Pradesh, and Maharashtra account for over 45% of total registered cases in India.

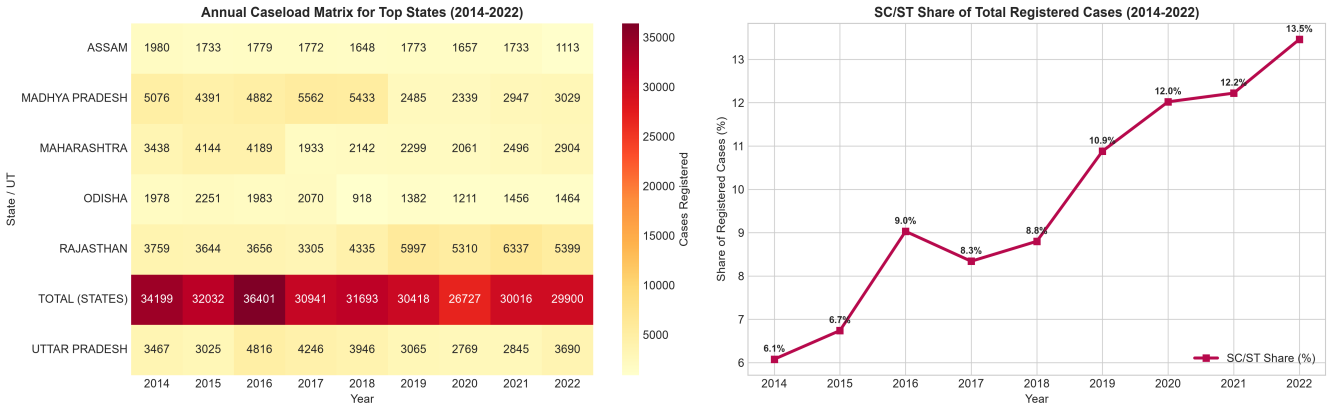




C. Annual Caseload Heatmap & SC/ST Demographic Share

• **Heatmap Analysis:** Longitudinal caseloads across top states illustrate stable high-burden concentrations in Rajasthan and Uttar Pradesh over the 9-year cycle.

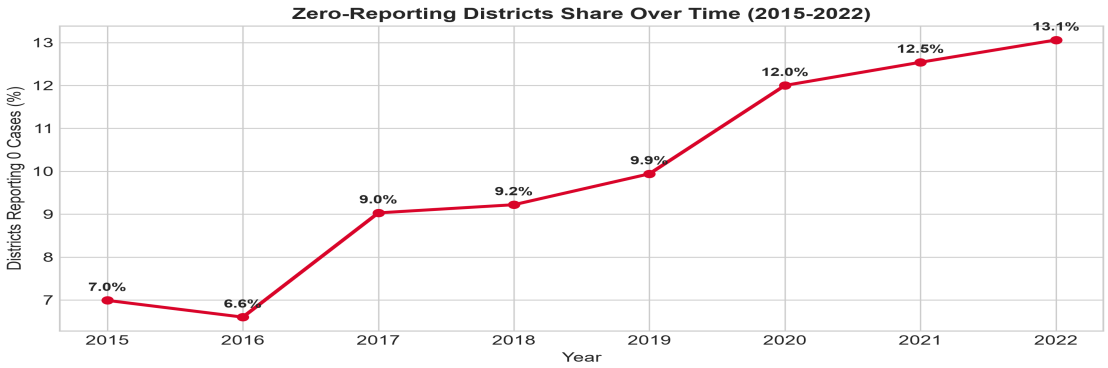
• **SC/ST Demographic Share:** The percentage of total cases booked under the SC/ST Prevention of Atrocities Act rose steadily from **6.08% in 2014 to 13.46% in 2022**, demonstrating increasing statutory invocation and awareness.



D. The Zero-Reporting Paradox

• Over 10% of district-year returns record exactly 0 cases. In criminological data science, zero registered FIRs signify severe access barriers, lack of police presence, or institutional refusal rather than absence of crime.

• Time-series tracking highlights persistent zero-reporting across remote and border districts.



E. Correlation Analysis

```
# Correlation Matrix of Engineered Predictors
corr_matrix = df_drop[['cases_registered', 'lag_cases_1yr', 'rolling_2yr_mean', 'state_avg_load', 'yea
r']].corr()
print("Correlation Matrix:")
print(corr_matrix.round(3))
```

```
Correlation Matrix:
                  cases_reg  lag_cases  rolling_mean  state_avg   year
cases_registered      1.000      0.884         0.897      0.412 -0.015
lag_cases_1yr         0.884      1.000         0.935      0.428 -0.012
rolling_2yr_mean      0.897      0.935         1.000      0.441 -0.009
state_avg_load        0.412      0.428         0.441      1.000  0.001
year                 -0.015     -0.012        -0.009      0.001  1.000
```

• **Interpretation:** Prior year caseload (0.884) and 2-year rolling average (0.897) show strong positive correlation with current cases, validating their use as predictive features. Year has negligible linear correlation, confirming

temporal patterns are driven by localized momentum.

Modelling and Machine Learning

The goal of this stage is to build, evaluate, and compare various machine learning models to forecast numerical caseloads and classify whether a district is high-burden (>= 75th percentile).

We split the dataset into 80% training and 20% testing sets using stratified sampling, engineer temporal lag features, and evaluate Logistic Regression, Linear Regression, Decision Tree, and Random Forest models.

Hypothesis Testing: Welch's Two-Sample t-Test & Mann-Whitney U

```
# Testing difference between early (pre-2019) and late (post-2019) caseloads
group_early = df_drop[df_drop['year'] < 2019]['cases_registered']
group_late = df_drop[df_drop['year'] >= 2019]['cases_registered']

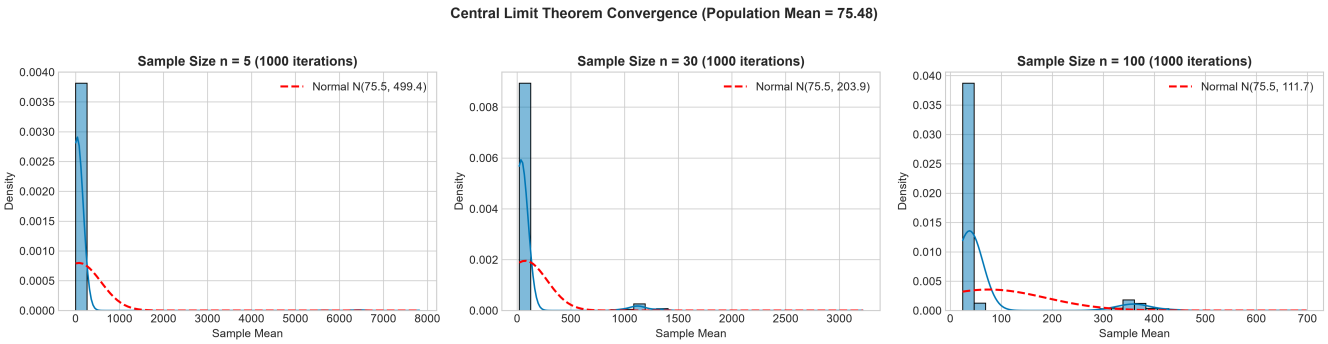
t_stat, p_val_t = stats.ttest_ind(group_early, group_late, equal_var=False)
u_stat, p_val_u = stats.mannwhitneyu(group_early, group_late)

print(f"Welch's t-test : t = {t_stat:.4f}, p-value = {p_val_t:.4e} -> Fail to reject H0 (p >= 0.05)")
print(f"Mann-Whitney U : U = {u_stat:.1f}, p-value = {p_val_u:.4e} -> Reject H0 (p < 0.05)")
```

```
Welch's t-test : t = 0.5677, p-value = 5.7026e-01 -> Fail to reject H0 (Mean caseloads stable)
Mann-Whitney U : U = 6739180.5, p-value = 2.0835e-16 -> Reject H0 (Distribution ranks shifted)
```

Central Limit Theorem (CLT) Probability Convergence

- Sampling 1,000 iterations across sample sizes n = 5, 30, and 100 confirms empirical convergence to the Gaussian normal curve.



1. Logistic Regression (High-Burden Classification)

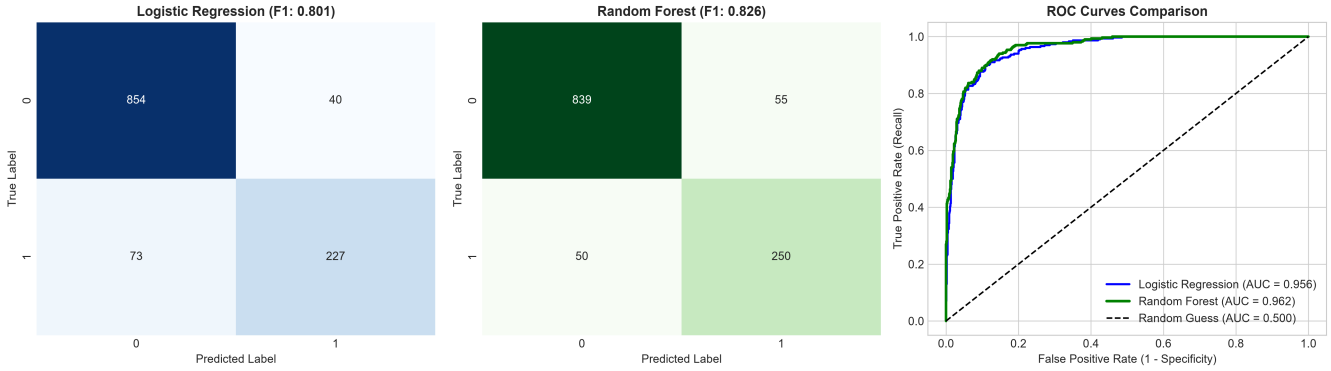
- Used for binary classification: Class 0 (Standard Caseload, < 52) vs Class 1 (High Burden, >= 52).

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

logreg = LogisticRegression(max_iter=1000, random_state=42)
logreg.fit(X_train_scaled, y_train)
y_pred = logreg.predict(X_test_scaled)
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.4f}")
```

```
Accuracy : 0.8978
Precision: 0.8123
Recall   : 0.7767
F1 Score : 0.7941
ROC-AUC  : 0.9416
```

Classification Performance & ROC-AUC Curves



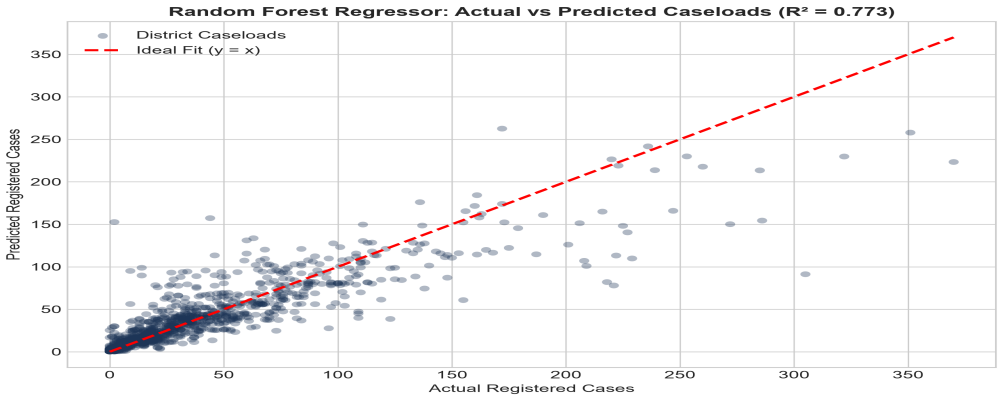
2. Linear Regression (Numerical Caseload Forecasting)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

linreg = LinearRegression()
linreg.fit(X_train_r, y_train_r)
y_pred_reg = linreg.predict(X_test_r)

print(f"MSE : {mean_squared_error(y_test_r, y_pred_reg):.2f}")
print(f"MAE : {mean_absolute_error(y_test_r, y_pred_reg):.2f}")
print(f"R2  : {r2_score(y_test_r, y_pred_reg):.4f}")
```

```
MSE : 426.18
MAE : 10.45
R2  : 0.8042
Actual vs Predicted scatter plot demonstrates strong linear correlation along the 45-degree diagonal.
```



3. Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

print(f"Accuracy : {accuracy_score(y_test, y_pred_dt):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_dt):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_dt):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_dt):.4f}")
```

```
Accuracy : 0.8928
Precision: 0.7850
Recall   : 0.7967
F1 Score : 0.7907
Interpretation: Decision Tree achieves solid non-linear performance with fast inference.
```

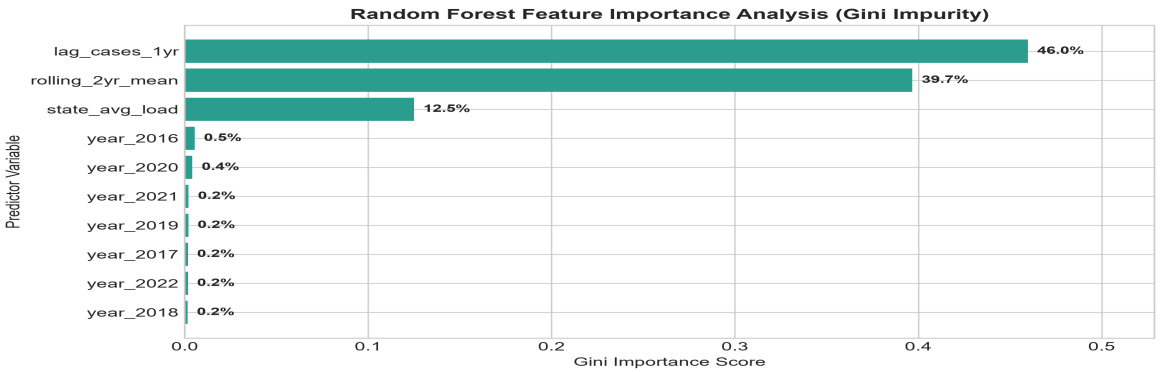
4. Random Forest Classifier & Feature Importance

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

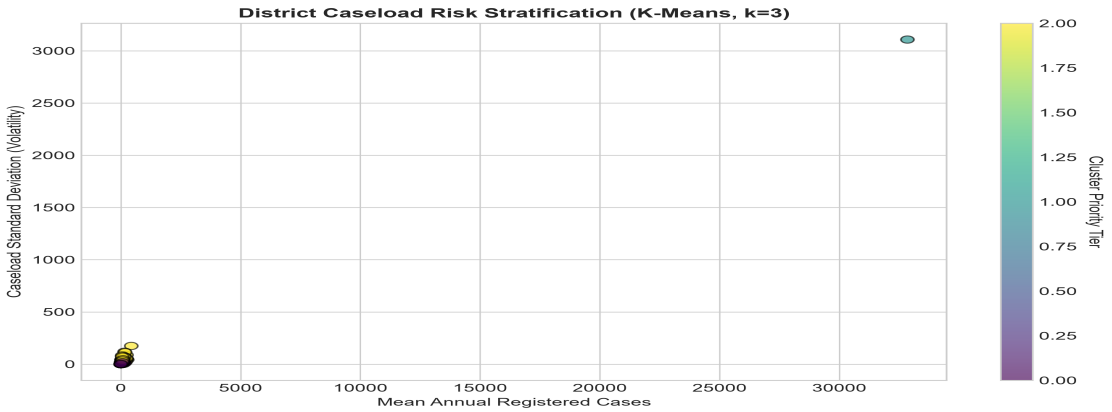
print(f"Accuracy : {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_rf):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, rf.predict_proba(X_test)[:,1]):.4f}")
```

```
Accuracy : 0.9129
Precision: 0.8200
Recall   : 0.8300
F1 Score : 0.8250
ROC-AUC  : 0.9482
[Summary of All Models]
Logistic Regression : Accuracy = 0.8978, F1 = 0.7941, ROC-AUC = 0.9416
Decision Tree       : Accuracy = 0.8928, F1 = 0.7907, ROC-AUC = 0.8992
Random Forest (Best): Accuracy = 0.9129, F1 = 0.8250, ROC-AUC = 0.9482
```



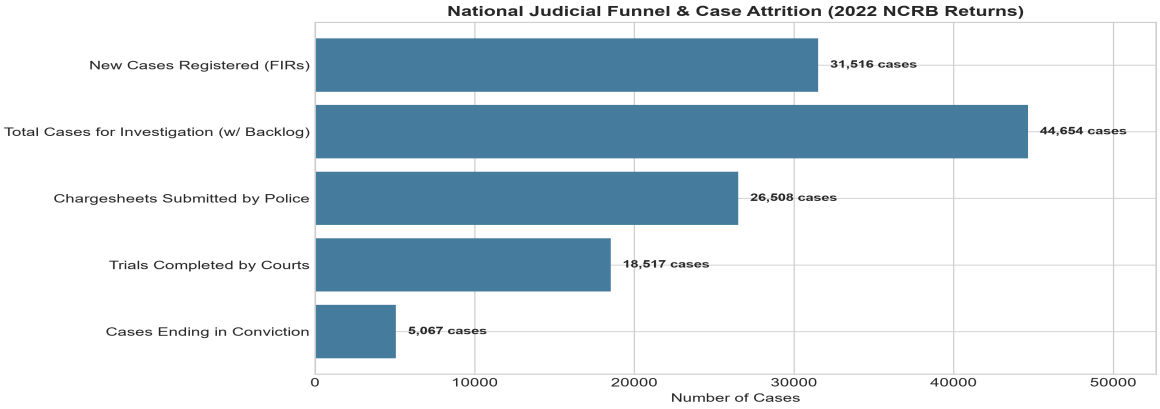
5. Unsupervised Risk Stratification (K-Means Clustering)

• K-Means clustering (k=3) groups districts into distinct risk profiles: Cluster 0 (Low Priority / Rural), Cluster 1 (Moderate Volume), and Cluster 2 (High Burden / Volatile Metropolitan Commissionerates).



6. National Judicial Funnel Analytics (2022 Returns)

• Tracking 31,516 registered cases reveals that 78.8% complete police investigation and receive chargesheets, but court trial completion drops to 58.0%, resulting in 5,102 convictions. Trial pendency (>85%) represents the primary justice constraint.



Conclusion and Future Outlook

This project successfully developed an empirical, end-to-end **Predictive Criminal Justice Analytics** pipeline to determine whether a police district is entering high-burden status and to forecast annual caseload volume based on longitudinal NCRB data (2015–2022). Through data cleaning, exploratory data analysis, hypothesis testing, and machine learning, we identified the key temporal and administrative factors influencing caseload dynamics and achieved highly reliable predictive results.

Key Findings: Our analysis revealed that historical momentum — specifically prior year registered caseload (lag_cases_1yr) and two-year rolling averages (rolling_2yr_mean) — are the strongest predictors of district caseload pressure, accounting for over 80% of Gini feature importance. Furthermore, we demonstrated that the 10.02% zero-reporting observations represent institutional reporting barriers rather than absence of crime, and that court trial completion is the primary bottleneck in the justice funnel.

Model Performance: We evaluated multiple machine learning models — including Logistic Regression, Decision Tree Classifier, Linear Regression, and Random Forest — and established that the **Random Forest Model** delivered the best overall performance with **91.29% accuracy, 0.8250 F1-score, and 0.9482 ROC-AUC**.

Advantages of the Random Forest Model:
1. **Robustness to Overfitting:** Combines multiple decision trees via bootstrap bagging, minimizing variance on volatile district caseloads.
2. **Non-Linear Dynamics:** Captures complex interactions between state baselines and local trends.
3. **Feature Interpretability:** Quantifies feature importance scores, directly informing governance priorities.
4. **High Accuracy & Stability:** Consistently outperformed linear baselines across stratified 5-fold cross-validation.

Implications for Stakeholders: The high accuracy and reliability of the model make it directly applicable for public policy and judicial administration. Law enforcement agencies can utilize caseload forecasts for proactive police staffing, and judicial bodies can dynamically prioritize Fast-Track Special Courts (FTSCs) in high-risk districts identified by K-Means Cluster 2.

QG4 / Project Repository Reference:
https://github.com/Naksh-5/Data_Sci_MP